Lab I1 - Drive DiffBot in NVIDIA Isaac Sim from ROS 2

Aim. Load the course robot into **Isaac Sim**, a physics (PhysX) and ray-tracing simulator, and drive it with a standard `/cmd_vel` command. Read `/odom`, `/tf` and `/joint_states` and compare the result with a simple kinematic simulator.

Time. 45 min (+ 10-minute download once, and 3 minutes for the very first start). **Needs.** Only this folder, an **NVIDIA RTX GPU** (tested: RTX 5050 Laptop, 8 GB, driver 592.82) and about 15 GB of disk for Isaac Sim, which `pixi run isaac-install` downloads once for all Isaac labs on the machine. By running the scripts you accept NVIDIA's Isaac Sim licence.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and an NVIDIA RTX GPU with a current driver; about 15 GB disk for Isaac Sim (shared by all Isaac labs, installed once into `C:\ix`); internet for the first install.

In this folder	
`isaac\`	the Isaac Sim scene scripts and ROS 2 nodes
`scripts\`	helper scripts and the automated tests
`code\`	reference solutions / helper programs for this lab
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

Isaac Sim needs Python 3.11, ROS 2 Jazzy on Windows needs 3.12, so they live in two environments and talk over DDS like two computers - both use ROS domain **78**. Isaac Sim's own ROS 2 bridge publishes and subscribes on its side.

```
Isaac Sim (isaac\, Python 3.11, installed in C:\ix)                ROS 2 (course environment,
Python 3.12)
 USD scene + PhysX physics + OmniGraph  <----- DDS, domain 78 ----->  pixi run isaac-ros ...,
isaac-drive ...
 /cmd_vel -> wheel joints ;  /odom /tf /joint_states /clock ->
```

- The robot is a **xacro** differential-drive model (expanded to `isaac\assets\diffbot.urdf`). Physics needed three additions that RViz does not: a 2 cm gap under the body so the wheels carry the weight, explicit link masses, and a centre of mass 3 cm behind the axle so the robot rests on its caster.
- An **OmniGraph** (Isaac's visual dataflow graph, built here from Python) is the whole driver: `/cmd_vel` -> `DifferentialController` (the standard differential-drive maths: `w_left = (v - w L/2)/r`, `w_right = (v + w L/2)/r`, r = 0.05, L = 0.30) -> wheel joints. The difference to Lab 3: here the **physics engine** moves the robot, so wheels can slip.
- Isaac Sim runs on **simulated time** (`/clock`), and not at wall-clock speed. The course helper `pixi run isaac-drive` counts *wall-clock* seconds, so the distance it drives depends on how fast your simulation runs (in window mode here: 1.32 m instead of 0.8 m). `code\drive_sim.py` counts `/clock` instead, so "0.2 m/s for 4 s" means 4 *simulated* seconds. Always pace experiments by `/clock`.

Procedure

1. Install Isaac Sim (once per machine, shared by all Isaac labs; about 10 GB into `C:\ix`, a short path because Windows' 260-character limit breaks Isaac's DLLs inside long folder names).

```
pixi run isaac-install
```

2. T1 - start the simulation with a window (first start about 3 min while shaders compile, later 15-60 s). Omit `--gui` to run without a window.

```
pixi run isaac-p1 --gui
```

3. T2 - what the simulator publishes. (`isaac-ros` is `ros2` on domain 78.)

```
pixi run isaac-ros topic list --no-daemon --spin-time 5
```

4. T2 - drive. Forward 0.2 m/s for 4 simulated seconds, then a left turn at 0.5 rad/s for 3 simulated seconds. Read the pose before and after each move.

```
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose
pixi run python code\drive_sim.py 0.2 0.0 4
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose
pixi run python code\drive_sim.py 0.0 0.5 3
pixi run isaac-ros topic echo /odom --no-daemon --once --spin-time 5 --field pose.pose
```

Compute the forward distance from the first two outputs and the heading change `2 * atan2(z, w)` from the last two. Expected: about 0.8 m (ideal: 0.2 x 4) and about 1.4-1.5 rad (ideal: 0.5 x 3 = 1.5). Physics makes it differ a little from the ideal (wheels need time to speed up and may slip); the automated test accepts 0.6-1.0 m and 1.1-1.9 rad.

For comparison, the wall-clock helper (`pixi run isaac-drive 0.2 0.0 4`) in window mode drove the robot 1.32 m and `isaac-drive 0.0 0.5 3` turned it 2.2 rad: the simulation ran about 1.5 times faster than real time.

5. T2 - the joints and the transform tree. The wheel angles and the frame `odom -> base_footprint` come from the simulator.

```
pixi run isaac-ros topic echo /joint_states --no-daemon --once --spin-time 5 --field name
pixi run isaac-ros topic echo /tf --no-daemon --once --spin-time 5
```

```
# Ctrl+C in T1 closes Isaac Sim
```

Observations (recorded)

Quantity	Value
`/odom` after 0.2 m/s for 4 simulated s	moved 0.82 m (`drive_sim.py`, window mode) and 0.86 m (automated test, headless); heading changed -0.006 rad
`/odom` after 0.5 rad/s for 3 simulated s	turned 1.37 rad (`drive_sim.py`) and 1.51 rad (test), moved 0.01 m
TF	`odom -> base_footprint` plus the transforms to every URDF link (`base_link`, wheels, caster, `lidar_link`)
Same drives with wall-clock `isaac-drive` (window mode)	1.32 m and 2.2 rad: the simulation ran about 1.5 x real time

Verification (starts its own headless Isaac Sim, 3-6 min; close T1 first)

```
pixi run test-isaac-p1
```

11 checks: `/clock`, `/odom`, TF to every link and `/joint_states` exist; the two drives move the robot 0.6-1.0 m and turn it 1.1-1.9 rad. Recorded: 11/11.

Questions

1. Why is the distance not exactly 0.80 m here, while a kinematic simulator would be exact? Name two effects a physics engine adds.
2. Why is `ROS_DOMAIN_ID` the same (78) in both programs, and why different from the other labs (default)?
3. Which of the three URDF additions would you remove first to see the robot tip over, and what do you expect?
4. How would the robot behave on a slippery floor? Which part of the graph would you change?
5. Why does the distance driven by `isaac-drive` depend on how fast the simulation runs, while `drive_sim.py` stays close to the ideal value? `r` n6. *(Extension)* Make the robot drive in a circle of radius 0.5 m: which `v` and `w`?

If it does not work

Symptom	Fix
"The filename or extension is too long"	the install is not in `C:\ix`; run `pixi run isaac-install`
Window opens but is black	wait; the first frames need shader compilation
`isaac-ros topic list` shows nothing	Isaac Sim is still starting (wait for `project 1 ready`)
Crash at start-up on a new NVIDIA driver	try the tested driver or run without `--gui`
`drive_sim.py` says "no /clock"	T1 not ready (wait for `project 1 ready`)